

Алгоритмы и структуры данных

Содержание

1. Некоторые обозначения	1
2. Хранение вершин	1
3. DFS	2
4. BFS	3
5. Топологическая сортировка	3
6. Поиск компонент связности в неориентированном графе	4
7. Компоненты слабой связности	5
8. Компоненты сильной связности	5
9. Конденсация графа	6
10. Поиск цикла в ориентированном графе	7
10.1. Восстановление ответа	8
10.2. Зачем 3 цвета?	8
11. Поиск эйлерова цикла	8
12. Поиск гамильтонового цикла при достаточных условиях	9
13. Алгоритм Дейкстры	9
13.1. Реализация алгоритма	9
13.2. Инвариант	11
14. Алгоритм Беллмана-Форда	11
14.1. Алгоритм	11



1. Некоторые обозначения

- Под переменной n будем иметь в виду количество вершин.
- Под переменной m будем иметь в виду количество ребер.
- Все вершины кодируются последовательно натуральными числами, начиная с нуля.
- По умолчанию имеется в виду **ориентированный** граф, если не сказано иначе.

2. Хранение вершин

Пусть последовательность натуральных чисел кодирует вершины. Тогда можно хранить граф внутри массива, причем двумя способами:

1. Матрица смежности:

```
vector<vector<bool>> g(n, vector<bool>(n));
g[2][4] = true; // добавление ребра 2 -> 4
bool adj = g[1][3]; // запрос вида "есть ли ребро 1 -> 3?"
```

2. Список смежности:

```
vector<vector<int>> g(n);
g[2].push_back(4); // добавление ребра 2 -> 4
for (int u : g[4]) { // обход всех вершин, смежных с вершиной 4
    cout << u << "\n";
}
```

3. Список ребер:

```
struct { int u, int w; } edge;
vector<edge> edges(m);
```

От выбора способа хранения зависит асимптотическая сложность обхода всего графа (вершин и исходящих из них ребер).

- Для матрицы смежности сложность составляет $O(n^2)$:

```
vector<vector<bool>> g(n, vector<bool>(n));
for (int v = 0; v < n; ++v) {
    for (int u = 0; u < n; ++u) {
        if (g[v][u]) {
            do_something(v, u);
        }
    }
}
```

- Для списка смежности сложность равна $O(n + m)$:

```
for (int v = 0; v < n; ++v) {
    for (int u : g[v]) {
        if (g[v][u]) {
            do_something(v, u);
        }
    }
}
```

```
}  
}
```

Обратим внимание на то, что второй цикл суммарно сделает m итераций. При этом может быть, что $n > m$, отсюда и сложность $O(n + m)$.

Список ребер полезен для некоторых алгоритмов (Беллмана-Форда, Краскала).

3. DFS

Depth-first search, обход в глубину.

Рекурсивно:

```
vector<vector<int>> g(n);  
vector<bool> used(n); // used[v] - посещена ли вершина v?  
  
void preorder(int v);  
void postorder(int v);  
  
void dfs(int v) {  
    used[v] = true;  
    preorder(v);  
    for (int u : g[v]) {  
        if (!used[u]) {  
            dfs(u);  
        }  
    }  
    postorder(v);  
}  
  
// main():  
  
dfs(1); // запуск DFS из вершины 1
```

С помощью стека:

```
vector<vector<int>> g(n);  
vector<bool> used(n);  
  
// main():  
  
stack<int> st;  
st.push(1)  
  
while (!st.empty()) {  
    int v = st.top();  
    st.pop()  
  
    used[v] = true;  
    do_something(v);  
    for (int u : g[v]) {
```

```

    if (!used[u]) {
        st.push(u);
    }
}
}

```

- Сложность по времени — $O(n + m)$
- Сложность по памяти — $O(n)$ для хранения `used` и стека (вызовов/вершин)

4. BFS

Breadth-first search, обход в ширину.

Алгоритм аналогичен DFS с помощью стека, но теперь вместо него используем очередь.

```

vector<vector<int>>> g(n);
vector<bool> used(n);

// main():

queue<int> q;
q.push(1)
used[1] = true;

while (!q.empty()) {
    int v = q.front();
    q.pop()

    do_something(v);
    for (int u : g[v]) {
        if (!used[u]) {
            used[u] = true;
            q.push(u);
        }
    }
}
}

```

- Сложность по времени — $O(n + m)$
- Сложность по памяти — $O(n)$ для `used` и очереди

5. Топологическая сортировка

Пусть нам дан DAG (Directed Acyclic Graph, направленный ациклический граф), и мы хотим упорядочить вершины так, чтобы ребра вели из младших по порядку вершин к старшим.

Идея такая: будем поочередно добавлять самые «глубокие» вершины. Для каждой непосещенной вершины v запустим DFS, где в postorder добавим v в массив. В результате получим обратный порядок, поэтому массив надо перевернуть.

```

vector<vector<int>> g(n);
vector<bool> used(n);
vector<int> order;

void dfs(int v) {
    used[v] = true;
    for (int u : g[v]) {
        if (!used[u]) {
            dfs(u);
        }
    }
    order.push_back(u);
}

// main():

for (int v = 0; v < n; ++v) {
    if (!used[v]) {
        dfs(v);
    }
}
reverse(order.begin(), order.end());

```

- Сложность по времени: $O(n + m)$
- Сложность по памяти: $O(n)$

6. Поиск компонент связности в неориентированном графе

Из каждой непосещенной вершины запускаем DFS/BFS, внутри которого помечаем вершины номером компоненты связности.

```

vector<vector<int>> g(n);
vector<bool> used(n);
int cur = 0; // номер текущей компоненты. В конце алгоритма равен их количеству
vector<int> comp(n, -1); // comp[v] - номер компоненты связности для вершины v

void dfs(int v) {
    used[v] = true;
    comp[v] = c;
    for (int u : g[v]) {
        if (!used[u]) {
            dfs(u);
        }
    }
}

// main():

for (int v = 0; v < n; ++v) {
    if (!used[v]) {
        dfs(v);
        ++c;
    }
}

```

```

    }
}
reverse(order.begin(), order.end());

```

Можно обойтись без массива `used`, делая вместо него проверку вида `if (comp[v] == -1)`.

- Сложность по времени: $O(n + m)$
- Сложность по памяти: $O(n)$

7. Компоненты слабой связности

Компоненты слабой связности в ориентированном графе эквивалентны компонентам связности в неориентированном.

Алгоритм примерно такой:

1. Делаем неориентированный граф, добавляя недостающие обратные ребра
2. Ищем компоненты связности в полученном неориентированном графе

8. Компоненты сильной связности

- Вершины v и u называются *сильно связными*, если из v можно попасть в u (любым возможным маршрутом в графе) и наоборот — из u в v .
- Компонента сильной связности — множество попарно сильно связных вершин.

Если так подумать, то сильная связность между вершинами — это отношение эквивалентности, а компоненты сильной связности — классы данного отношения.

Тривиальный алгоритм выглядит так:

0. Пусть $\text{reach}(v) = \{u \mid (v, u) \in E\}$ — множество вершин, которые достижимы из v в u .
1. С помощью любого обхода найдем $\text{reach}(v)$ для каждого v .
2. Для каждого v найдем $\text{reach}^{-1}(v) = \{u \mid u \in \text{reach}(v)\}$ — множество вершин, которые достижимы из v в $u \in \text{reach}(v)$
3. Компонентой сильной связности для вершины v будет являться $\text{reach}(v) \cap \text{reach}^{-1}(v)$.

Этот алгоритм работает за $O(nm)$. Для каждой из n вершин мы запускаем обходы за $O(n + m)$.

Однако можно найти компоненты сильной связности за $O(n + m)$:

1. Делаем топологическую сортировку
2. Строим транспонированный граф
3. С помощью любого обхода запускаем DFS в порядке топсорта, внутри которого помечаем вершины номерами компоненты связности.

Реализация:

```

vector<vector<int>>> g(n);
vector<vector<int>>> t(n);
vector<bool> used(n); // используется для топсорта
vector<int> order; // порядок топсорта
int c = 0; // текущий номер компоненты
vector<int> comp(n, -1); // comp[v] - номер компоненты для вершины v

```

```

void dfs_topsort(int v) {
    used[v] = true;
    for (int u : g[v]) {
        if (!used[u]) {
            dfs(u);
        }
    }
    order.push_back(u);
}

void dfs_comp(int v) {
    comp[v] = c;
    for (int u : t[v]) { // <--- обходим транспонированный граф!
        if (comp[u] == -1) {
            dfs(u);
        }
    }
}

// main():

// 1. топсорт
for (int v = 0; v < n; ++v) {
    if (!used[v]) {
        dfs_topsort(v);
    }
}
reverse(order.begin(), order.end());

// 2. транспонирование
for (int v = 0; v < n; ++v) {
    for (int u : g[v]) {
        t[u].push_back(v); // обратите внимание на порядок u -> v
    }
}

// 3. помечание компонент связности
for (int v = 0; v < n; ++v) {
    if (comp[v] == -1) {
        dfs_comp(v);
        ++c;
    }
}

```

Объяснение того, почему этот алгоритм работает, можно найти в [статье от USACO](#).

Существует алгоритм Тарьяна, который позволяет найти компоненты сильной связности всего за один проход DFS вместо двух. Рассматривать его не буду.

9. Конденсация графа

Суть в том, чтобы «сжать» целую компоненту слабой/сильной связности в одну вершину. Это и будет называться *конденсацией графа*.

Запустим любой обход. Если встречается ребро, переходящее проходящее между вершинами из разных компонент связности, то добавляем в конденсацию ребро между этими компонентами.

```
vector<vector<int>> g(n);
vector<bool> used(n);

// пусть мы уже нашли компоненты сильной/слабой связности
int cnt; // количество компонент
vector<int> comp; // comp[v] - номер компоненты для вершины v

// список смежности конденсации графа
// хотя правильнее сказать множество смежности; он тут будет удобнее
vector<unordered_set<int>> condensation(c);

void dfs(int v) {
    used[v] = true;
    for (int u : g[v]) {
        if (comp[v] != comp[u]) {
            condensation[comp[v]].insert(comp[u]);
        }
        if (!used[u]) {
            dfs(u);
        }
    }
}

for (int v = 0; v < n; ++v) {
    if (!used[v]) {
        dfs(v);
    }
}
```

10. Поиск цикла в ориентированном графе

Введем три цвета: белый, серый, черный. Изначально все вершины белые. Для каждой белой вершины запускаем DFS. Внутри него, в preorder помечаем вершину серым, а в postorder черным. Если смежная вершина серая — цикл найден.

```
const int WHITE = 0, GRAY = 1, BLACK = 2;
vector<vector<int>> g(n);
vector<int> color(n, WHITE);
bool has_cycle = false;

void dfs(int v) {
    color[v] = GRAY;
    for (int u : g[v]) {
        if (color[u] == GRAY) {
            has_cycle = true;
        } else if (color[u] == WHITE) {
            dfs(u);
        }
    }
    color[v] = BLACK;
}
```



```

    }
}
color[v] = BLACK;
}

// main():

for (int v = 0; v < n; ++v) {
    if (color[v] == WHITE) {
        dfs(v);
    }
}
}

```

10.1. Восстановление ответа

Можно хранить массив предков и обновлять его в preorder. Когда нашли цикл, проходимся по этому массиву, пока не дойдем обратно до вершины v .

10.2. Зачем 3 цвета?

Проиллюстрирую на примере.

Рассмотрим граф с вершинами A, B, C и ребрами $A \rightarrow B, B \rightarrow C, A \rightarrow C$. Заметим, что при DFS из A мы попадем в черную вершину C . При этом это не цикл!

11. Поиск эйлерова цикла

- *Эйлеров путь* — путь, в котором каждое ребро встречается только один раз.
- *Эйлеров цикл* — эйлеров путь, являющийся циклом :)

Теорема. Эйлеров цикл существует тогда и только тогда, когда:

1. Степень каждой вершины четная
2. Граф связный

```

vector<unordered_set<int>> g(n); // множество смежности для удобства
vector<int> path;

// алгоритм поиска эйлерова цикла
// работает корректно, если выполнены достаточные условия
void euler(int v) {
    while (!g[v].empty()) {
        int u = g[v][0];
        g[v].erase(u);
        // если граф неориентированный:
        // g[u].erase(v);
        euler(u);
    }
    path.push_back(v);
}

// main():

```

```

// 1-ое условие: deg(v) четный
for (int v = 0; v < n; ++v) {
    if (g[v].size() % 2 != 0) {
        cout << "impossible\n";
        return 0;
    }
}

// 2-ое условие: граф связный
if (!is_connected()) {
    cout << "impossible\n";
    return 0;
}

euler(0); // или от любой другой вершины

// теперь в path хранится эйлеров путь

```

12. Поиск гамильтонового цикла при достаточных условиях

- *Гамильтонов путь* — путь, проходящий через каждую вершину ровно один раз
- *Гамильтонов цикл* — замкнутый гамильтонов путь
- *Гамильтонов граф* — граф, содержащий гамильтонов цикл

Теорема (Дирака). Если $n \geq 3$ и для любой вершины v выполняется $\deg(v) \geq \frac{n}{2}$, то граф гамильтонов.

Теорема (Оре). Если $n \geq 3$ и для любых двух несмежных вершин v и u выполняется $\deg(v) + \deg(u) \geq n$, то граф гамильтонов.

Алгос [здесь](#), мне лень.

13. Алгоритм Дейкстры

Алгоритм поиска кратчайшего пути от одной вершины до всех остальных во взвешенном графе. Важное условие — все ребра неотрицательного веса.

13.1. Реализация алгоритма

Первый вариант — линейно искать вершину с наименьшим $d[v]$. Работает за $O(n^2)$.
Полезен $m \approx n^2$.

```

def dijkstra(s):
    used = [False] * n
    d = [INF] * n
    d[s] = 0

    for i in range(n):
        v = -1
        for j in range(n):
            if not used[j] and (v == -1 or d[j] < d[v]):

```

```

    v = j

    used[v] = True

    for u, w in g[v]:
        d[u] = min(d[u], d[v] + w)

```

Второй вариант — использовать дерево поиска. Ключом будет служить кратчайшее расстояние до вершины. В таком случае алгоритм будет работать за $O(m \log n)$ (m раз вставляем/удаляем вершины за $\log n$). Полезно, когда $m \approx n$. Заметим, что при $m \approx n^2$ данный вариант будет работать за $O(n^2 \log n)$, так что линейный поиск тоже бывает полезен.

```

def dijkstra(s):
    d = [INF] * n
    d[s] = 0

    q = set()
    q.add((d[s], s))

    while q:
        _, v = q.pop()

        for u, w in g[v]:
            a.remove((d[u], u))
            d[u] = min(d[u], d[v] + w)
            a.add((d[u], u))

```

Третий вариант — использовать приоритетную очередь. Асимптотически такой же, как и с деревом поиска.

Если под рукой реализация приоритетной очереди, не поддерживающая операцию `decrease_key` (например, бинарная куча), то можно просто пропускать вершины с неактуальным расстоянием.

```

from heapq import heappush, heappop

def dijkstra(s):
    d = [INF] * n
    d[s] = 0

    q = []
    heappush(q, (0, s))

    while q:
        w_cur, v = heappop(q)
        if w_cur > d[v]:
            continue

        for u, w in g[v]:

```

```
d[u] = min(d[u], d[v] + w)
heappush(q, (d[u], u))
```

13.2. Инвариант

- Если вершина v не посещена, то $d[v]$ — длина кратчайшего пути, полученного из посещенных вершин.
- Если вершина v посещена, то $d[v]$ — окончательная длина кратчайшего пути.

Корректность алгоритма доказывается на данном инварианте как индукционным предположением.

14. Алгоритм Беллмана-Форда

Алгоритм поиска кратчайшего пути из вершины s во все остальные. Поддерживает ребра отрицательного веса и позволяет найти *цикл отрицательного веса* — цикл, сумма ребер которого отрицательна.

14.1. Алгоритм

Будем делать строить двумерное ДП (причем динамика вперед). Пусть $d[k][v]$ — кратчайшее расстояние от начальной s до v за не более, чем k ребер.

- База:

$$d[k][v] = \begin{cases} 0 & m = 0, v = s \\ +\infty & \text{в остальных случаях} \end{cases}$$

- Переход:

$$d[k+1][u] = \min \begin{cases} d[k+1][u] \\ d[k][v] + w(v, u), \quad (v, u) \in E \end{cases}$$

Для корректной работы алгоритма достаточно $m - 1$ строк.

Заметим, что можно выкинуть строки динамики. В итоге получаем одномерное ДП $d[v]$, которое надо прорелаксировать $m - 1$ раз. Также заметим, что в данном алгоритме нам удобно хранить граф в виде списка ребер.

Реализация:

```
g = [(1, 2, 5), (2, 3, 10), ...] # список ребер

def bellman_ford(s):
    d = [1e18] * n
    d[s] = 0

    for k in range(n - 1):
        for u, v, w in g:
            d[v] = min(d[v], d[u] + w) # TODO
```

Асимптотика очевидна: $O(nm)$.